Algorithmes sur les arbres binaires. Correction.

I) RAPPELS. GENERALITES :

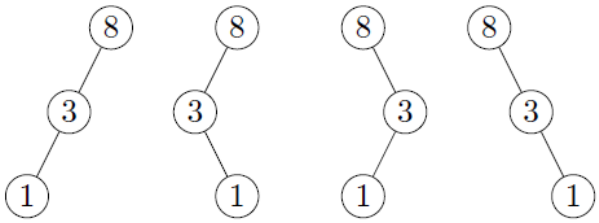
1°) Généralités :

Nous avons déjà vu la notion d'arbre, une structure qui n'est pas linéaire comme les listes, les piles ou les files, mais souvent rangée dans la catégorie des structures hiérarchiques. Pour simplifier nous supposerons que les données rangées dans les nœuds de la structure sont des entiers.
Il existe différents types d'arbres, et la nomenclature n'est pas universelle.
Par exemple, on peut trouver la définition suivante : **un arbre est un graphe connexe sans cycle**. Les arbres ainsi définis ne sont pas enracinés, ils n'ont pas une arité fixe (un nœud peut avoir un nombre variable de fils), et même pour un nœud possédant deux fils, il n'y a pas d'ordre sur les fils, donc ni fils gauche ni fils droit. Nous n'étudions pas ici ce type d'arbres.

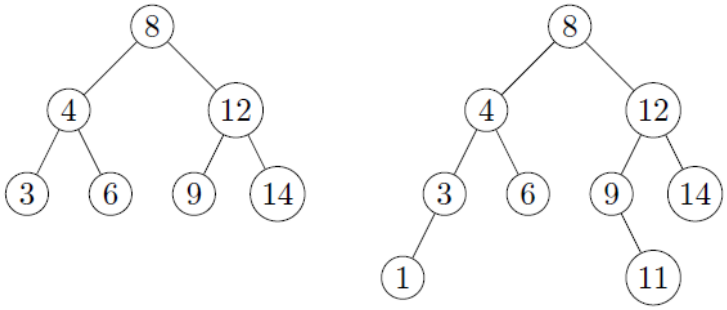
2°) Définition :

Un ***arbre binaire*** est défini de la façon suivante : **ou bien il est vide**, **ou bien il est constitué d'un nœud *racine*** qui possède deux fils, **un fils gauche et un fils droit, qui sont tous les deux des arbres binaires.**
Il s'agit donc d'une définition **récursive**.

Dans la figure suivante, tous les arbres dessinés sont deux à deux distincts, ce qui confirme qu'il faut distinguer fils droit et fils gauche, et qu'un des fils peut être vide.



Considérons les deux arbres de la figure suivante.

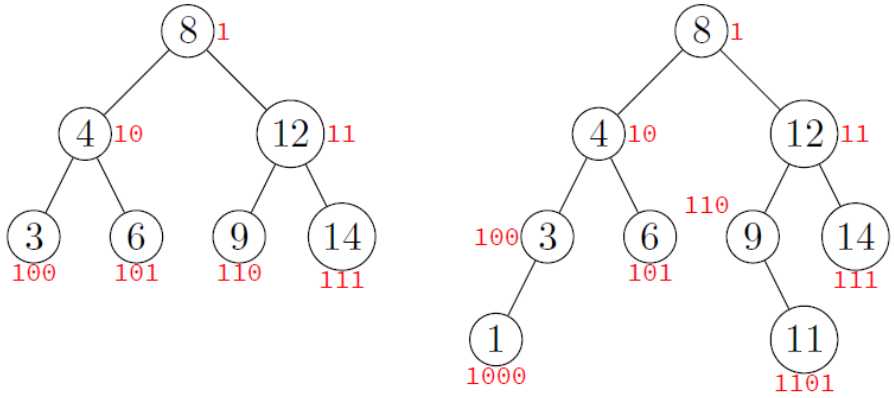


Le premier a pour racine un nœud contenant l'entier 8, la racine de son fils gauche contient l'entier 4, la racine de son fils droit contient l'entier 12.
Remarque : deux nœuds différents peuvent contenir la même valeur.
Le nombre de nœuds d'un arbre binaire s'appelle sa *taille*. Ainsi la taille de l'arbre de gauche est égale à 7, celle de l'arbre de droite est égale à 9.
On dit que la racine est à *profondeur* 0, les racines de ses fils à profondeur 1, etc.
Ainsi, pour l'arbre de droite, il y a un nœud à profondeur 0 (c'est 8), deux nœuds à profondeur 1 (ce sont 4 et 12), quatre nœuds à profondeur 2 (ce sont 3, 6, 9 et 14), et deux nœuds à profondeur 3 (1 et 11).
La profondeur maximale d'un nœud de l'arbre s'appelle la *hauteur* de l'arbre.
Ainsi l'arbre de gauche est de profondeur 2 et l'arbre de droite de profondeur 3.
Remarquons que la hauteur d'un arbre vide n'est pas définie.
On peut également définir la hauteur d'un arbre comme la longueur (en nombre d'arêtes) du plus long chemin allant d'un nœud à la racine.

3°) Un étiquetage intéressant :

On peut étiqueter les nœuds d'un arbre de la façon suivante :
* la racine porte l'étiquette 1 ;
* si un nœud quelconque porte l'étiquette x...yz, la racine de son éventuel fils gauche porte l'étiquette x...yz0 et la racine de son éventuel fils droit l'étiquette x...yz1.

On observe que les étiquettes des nœuds de profondeur p sont constituées de p + 1 bits, et sont deux à deux distinctes.
On en déduit que toutes les étiquettes des nœuds d'un même arbre sont deux à deux distinctes.
Les étiquettes peuvent être vues comme les écritures de numéros en base 2 : on a ainsi numéroté les différents nœuds de l'arbre.
Voici le résultat de cet étiquetage sur les deux arbres que nous avons déjà considérés.



Dans le cas de l'arbre de gauche, si on traduit en décimal les étiquettes des nœuds, on constate qu'on les a simplement numérotés de 1 à 7, de haut en bas et de gauche à droite.
Pour l'arbre de droite, on a pioché dans les numéros de 1 à 15, sans utiliser les numéros 9, 10, 11, 12, 14, 15 qui correspondent à des nœuds absents.
Un arbre binaire de hauteur h tel qu'à chaque profondeur p tous les nœuds sont présents (il y en a 2^p) est dit *complet*.
C'est le cas de l'arbre de gauche de la figure ci-dessus.
Un arbre binaire qui est complet sauf à la profondeur p = h où tous les nœuds sont à gauche sera étiqueté avec tous les entiers de 1 à n, sans « trou » dans la numérotation : on dira que c'est un arbre bien tassé. L'arbre de droite de la figure n'est pas bien tassé : il manque quatre nœuds (le fils droit de 3, les deux fils de 6 et le fils gauche de 9).

4°) Relation entre taille et profondeur :

Pour une taille n fixée, la hauteur maximale d'un arbre binaire est h = n − 1, qu'on obtient avec des arbres « filiformes » comme ceux de la première figure que nous avons dessinée.
Les arbres de taille n de hauteur minimale sont les arbres bien tassés. On a vu qu'alors l'étiquetage proposé utilise tous les entiers de 1 à n. Or l'étiquette du dernier nœud, de profondeur p = h, est l'écriture binaire de n, et s'écrit — on l'a déjà remarqué — avec p + 1 bits. Donc pour cet arbre, on a $h = \lfloor \log_2(n) \rfloor$. (Rem : $\lfloor \log_2(n) \rfloor$ correspond à la partie entière inférieure de $\log_2(n)$).
En effet, le logarithme en base 2 d'un entier n n'est autre que le nombre de bits nécessaire à son écriture en base 2 diminué d'une unité.
On a donc pour n'importe quel arbre binaire de hauteur h et de taille n les inégalités : $\lfloor \log_2(n) \rfloor <= h <= n - 1$.

II) HAUTEUR ET TAILLE D'UN ARBRE BINAIRE :

1°) Notations utilisées dans les algorithmes :

Soit un **arbre T** : **T.racine** correspond au nœud racine de l'arbre T.
Soit un **nœud x** :
x.gauche correspond au **sous-arbre gauche** du **nœud x ;**
x.droit correspond au **sous-arbre droite** du **nœud x ;**
x.clé correspond à la **clé** du **nœud x.**

Il faut noter que si le **nœud x** est une feuille, **x.gauche** et **x.droite** sont des arbres vides (NIL).

2°) Calculer la hauteur d'un arbre :

■ Principe :
- un arbre vide est de hauteur 0 ;
- un arbre non vide a pour hauteur 1 + la hauteur maximale entre ses fils.

■ **Algorithme en pseudo-code :**

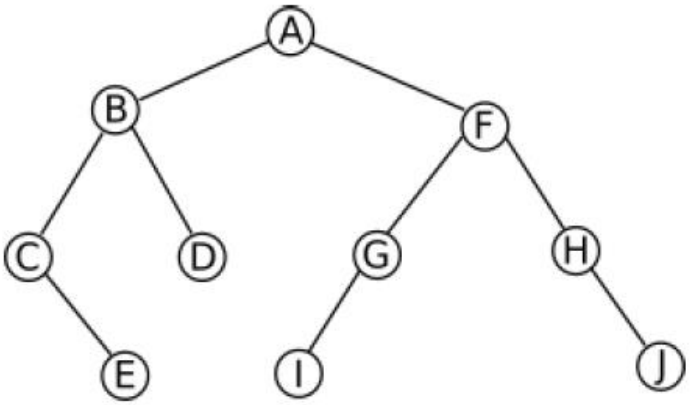
```
VARIABLE
T : arbre
x : noeud

DEBUT
HAUTEUR(T) :
   si T ≠ NIL :
      x ← T.racine
      renvoyer 1 + max(HAUTEUR(x.gauche), HAUTEUR(x.droit))
   sinon :
      renvoyer 0
   fin si
FIN
```

Remarque : la fonction max renvoie la plus grande valeur des 2 valeurs passées en paramètre, par exemple : max(5,6) renvoie 6.

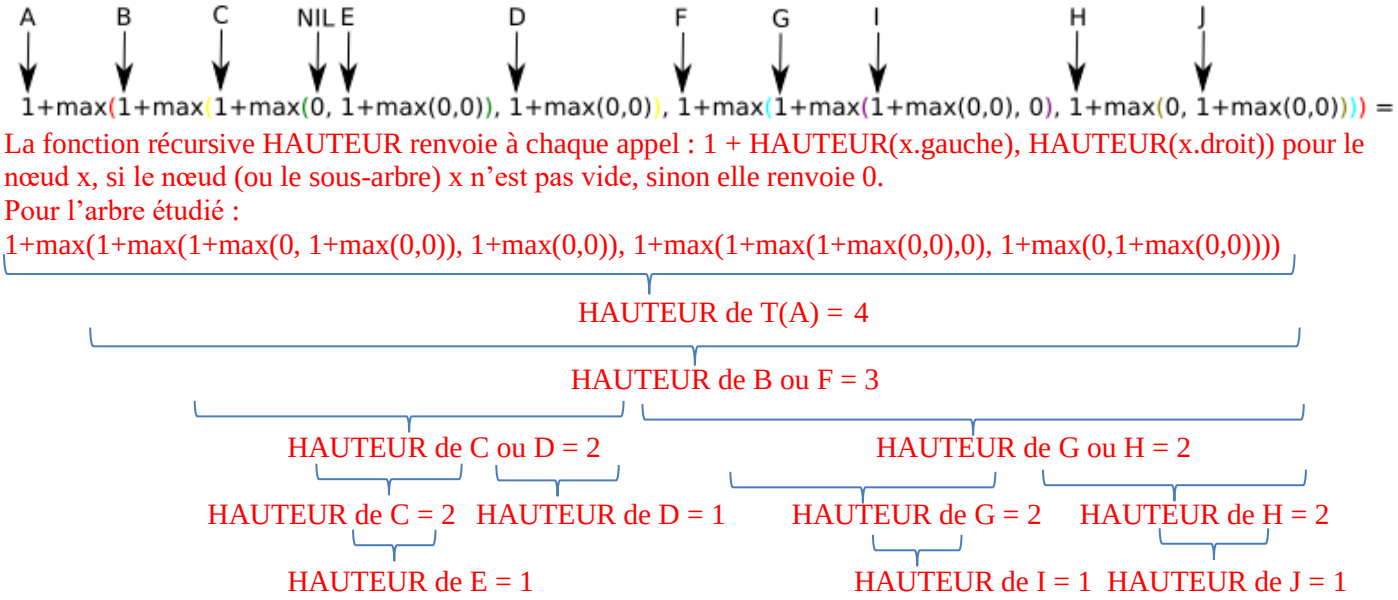
a) Quelle est la particularité de la fonction HAUTEUR() ?

C'est une fonction récursive.

b) Appliquer l'algorithme ci-dessus sur l'arbre binaire du paragraphe ci-dessous afin de calculer sa hauteur en détaillant votre raisonnement.



On pourra en particulier compléter le calcul sous la forme suivante :



E : pas de sous-arbre D : pas de sous-arbre I : pas de sous-arbre J : pas de sous-arbre

Donc pour ces nœuds, la fonction renvoie : 1 + max(0,0), soit 1.

On calcule HAUTEUR en partant du bas, et on obtient HAUTEUR = 4.

3°) Calculer la taille d'un arbre (nombre de nœuds) :

■ Principe :

- si l'arbre est vide : renvoyer 0 ;
- sinon renvoyer 1 plus la somme du nombre de nœuds des sous-arbres.

■ **Algorithme en pseudo-code :**

```
VARIABLE
T : arbre
x : noeud

DEBUT
TAILLE(T) :
   si T ≠ NIL :
      x ← T.racine
      renvoyer 1 + TAILLE(x.gauche) + TAILLE(x.droit)
   sinon :
      renvoyer 0
   fin si
FIN
```

a) Quelle est la particularité de la fonction TAILLE() ?

C'est une fonction récursive.

b) En vous inspirant du paragraphe précédent, appliquer l'algorithme ci-dessus sur l'arbre binaire du paragraphe précédent afin de calculer sa taille en détaillant votre raisonnement.

La fonction TAILLE renvoie 1 + TAILLE(x.gauche) + TAILLE(x.droit) pour le nœud x, si le nœud (ou le sous-arbre) x n'est pas vide, sinon elle renvoie 0.

On a donc :

TAILLE(T) = TAILLE(A) = 1 + TAILLE(B) + TAILLE(F) = 10

TAILLE(B) = 1 + TAILLE(C) + TAILLE(D) = 4 et TAILLE(F) = 1 + TAILLE(G) + TAILLE(H) = 5

TAILLE(C) = 1 + TAILLE(E) + TAILLE(0) = 2

TAILLE(D) = 1 + TAILLE(0) + TAILLE(0) = 1

TAILLE(G) = 1 + TAILLE(I) + TAILLE(0) = 2

TAILLE(H) = 1 + TAILLE(J) + TAILLE(0) = 2

TAILLE(E) = 1 + TAILLE(0) + TAILLE(0) = 1

TAILLE(I) = 1 + TAILLE(0) + TAILLE(0) = 1

TAILLE(J) = 1 + TAILLE(0) + TAILLE(0) = 1

Commencez toujours par le bas !

En appliquant le même type de raisonnement qu'au paragraphe précédent, on trouve donc : TAILLE(T) = 10.

III) PARCOURS EN PROFONDEUR D'UN ARBRE BINAIRE :

1°) Parcourir un arbre dans l'ordre infixe :

■ Principe :

Dans un **parcours infixe** (inorder traversal), chaque nœud est visité après son enfant gauche mais avant son enfant droit. Le parcours infixe affiche la racine après avoir traité le sous-arbre gauche, après traitement de la racine, on traite le sous-arbre droit (c'est donc un parcours de type G R D : Gauche Racine Droite).

■ **Algorithme en pseudo-code :**

```
VARIABLE
T : arbre
x : noeud

DEBUT
PARCOURS-INFIXE(T) :
   si T ≠ NIL :
      x ← T.racine
      PARCOURS-INFIXE(x.gauche)
      affiche x.clé
```

```
    PARCOURS-INFIXE(x.droit)
  fin si
FIN
```

a) Quelle est la particularité de la fonction PARCOURS-INFIXE() ?
C'est une fonction récursive.
b) Dans quel ordre est parcouru l'arbre binaire du paragraphe précédent par la fonction PARCOURS-INFIXE() ?
On obtient donc : C, E, B, D, A, I, G, F, H, J.

2°) Parcourir un arbre dans l'ordre préfixe :

■ Principe :

Dans un **parcours préfixe** (preorder traversal), chaque nœud est visité avant que ses enfants soient visités. Cela signifie que l'on affiche la racine de l'arbre, on parcourt tout le sous-arbre de gauche, une fois qu'il n'y a plus de sous-arbre gauche on parcourt les éléments du sous-arbre droit. Ce type de parcours peut être résumé en trois lettres : R G D (pour Racine Gauche Droit).

■ **Algorithme en pseudo-code :**

```
VARIABLE
T : arbre
x : noeud

DEBUT
PARCOURS-PREFIXE(T) :
  si T ≠ NIL :
    x ← T.racine
    affiche x.clé
    PARCOURS-PREFIXE(x.gauche)
    PARCOURS-PREFIXE(x.droit)
  fin si
FIN
```

a) Quelle est la particularité de la fonction PARCOURS-PREFIXE() ?
C'est une fonction récursive.
b) Dans quel ordre est parcouru l'arbre binaire du paragraphe précédent par la fonction PARCOURS-PREFIXE() ?
On obtient : A, B, C, E, D, F, G, I, H, J.

3°) Parcourir un arbre dans l'ordre suffixe (ou postfixe) :

■ Principe :

Dans un parcours suffixe (ou postfixe pour postorder traversal), chaque nœud est visité après que ses enfants sont visites. Le parcours postfixe effectue schématiquement le parcours suivant : sous-arbre gauche, sous arbre-droit puis la racine, c'est donc un parcours G D R (Gauche Droite Racine).

■ **Algorithme en pseudo-code :**

```
VARIABLE
T : arbre
x : noeud

DEBUT
PARCOURS-SUFFIXE(T) :
  si T ≠ NIL :
    x ← T.racine
    PARCOURS-SUFFIXE(x.gauche)
    PARCOURS-SUFFIXE(x.droit)
    affiche x.clé
  fin si
FIN
```

a) Quelle est la particularité de la fonction PARCOURS-SUFFIXE() ?
C'est une fonction récursive.

b) Dans quel ordre est parcouru l'arbre binaire du paragraphe précédent par la fonction PARCOURS-SUFFIXE() ?
On obtient : E, C, D, B, I, G, J, H, F, A.

Le choix du parcours infixe, préfixe ou suffixe dépend du problème à traiter, on pourra retenir pour les parcours préfixe et suffixe (le cas du parcours infixe sera traité un peu plus loin) que :

- dans le cas du parcours préfixe, un nœud est affiché avant d'aller visiter ces enfants ;
- dans le cas du parcours suffixe, on affiche chaque nœud après avoir affiché chacun de ses fils.

IV) PARCOURS EN LARGEUR D'UN ARBRE BINAIRE :

■ Principe :
L'**algorithme de parcours en largeur** (ou BFS, pour *Breadth First Search* en anglais) permet le parcours d'un arbre de la manière suivante : on commence par explorer un nœud source, puis ses successeurs, puis les successeurs non explorés des successeurs, etc. A partir d'un nœud source S, il liste d'abord les voisins de S pour ensuite les explorer un par un. Ce mode de fonctionnement utilise donc une file dans laquelle il prend le premier sommet et place en dernier ses voisins non encore explorés. Le principe de l'algorithme est le suivant :
1. Mettre le nœud source dans la file.
2. Retirer le nœud du début de la file pour le traiter.
3. Mettre tous les voisins non explorés dans la file (à la fin).
4. Si la file n'est pas vide reprendre à l'étape 2.

■ **Algorithme en pseudo-code :**

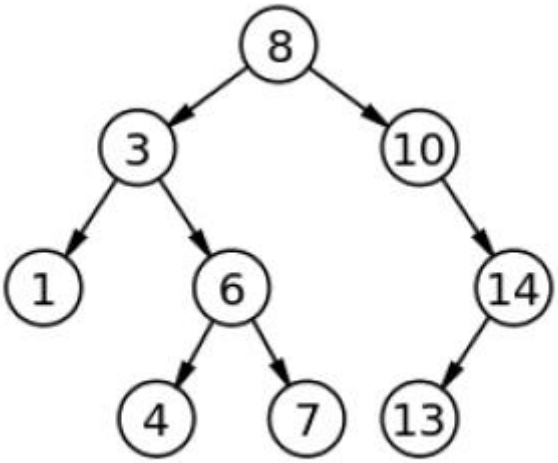
```
VARIABLE
T : arbre
Tg : arbre
Td : arbre
x : noeud
f : file (initialement vide)

DEBUT
PARCOURS-LARGEUR(T) :
   enfiler(T.racine, f) //on place la racine dans la file
   tant que f non vide :
      x ← defiler(f)
      affiche x.clé
      si x.gauche ≠ NIL :
         Tg ← x.gauche
         enfiler(Tg.racine, f)
      fin si
      si x.droit ≠ NIL :
         Td ← x.droite
         enfiler(Td.racine, f)
      fin si
   fin tant que
FIN
```

a) Dans quel ordre est parcouru l'arbre binaire du paragraphe précédent par la fonction PARCOURS-LARGEUR() ?
On obtient : A, B, F, C, D, G, H, E, I, J.
b) Selon vous, pourquoi parle-t-on de parcours en largeur ?
Le parcours en largeur consiste à parcourir l'arbre niveau par niveau. Les nœuds de niveau 0 sont d'abord parcourus puis les nœuds de niveau 1 et ainsi de suite. Dans chaque niveau, les nœuds sont parcourus de la gauche vers la droite.
Remarque : on utilise une file (FIFO) pour cet algorithme de parcours en largeur. En outre, cet algorithme n'utilise pas de fonction récursive.

V) ARBRE BINAIRE DE RECHERCHE :

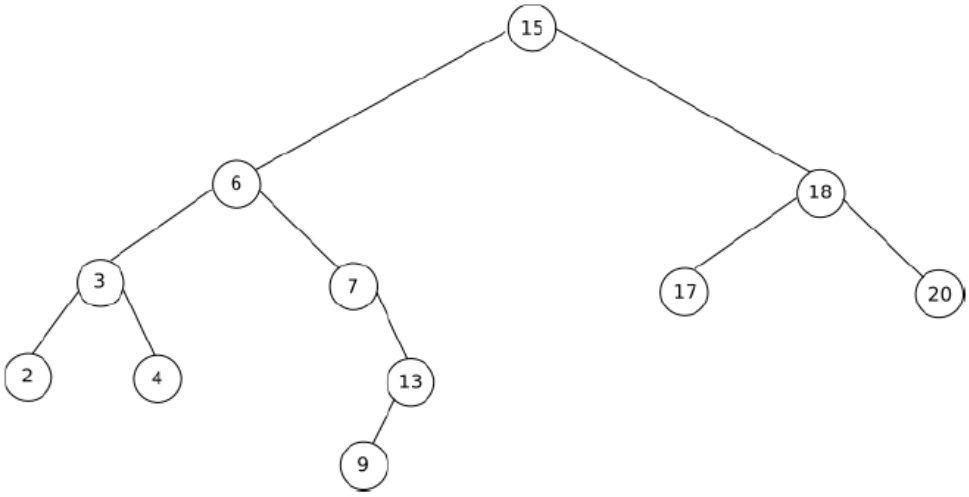
1°) Notion d'arbre binaire de recherche :

Les éléments stockés dans un **arbre binaire de recherche ABR** (en anglais, **B**inary **S**earch **T**ree ou **BST)** doivent posséder une **relation d'ordre totale**, c'est-à-dire qu'il doit exister une manière de dire si un élément est plus grand ou plus petit qu'un autre. Par exemple, on peut trier des nombres par valeur ou des personnes par ordre alphabétique de leur nom de famille.
Le premier élément inséré dans l'arbre devient la racine. Ensuite, il suffit de mettre à gauche les éléments plus petits et à droite les éléments plus grands. C'est cette particularité qui rend les BST intéressants : la plupart des opérations réalisées sur les arbres binaires de recherche ont une complexité temporelle logarithmique O(log n) dans le cas moyen. Cela est dû au fait que, grâce à la relation d'ordre liant les valeurs, on peut naviguer dans l'arbre avec une logique rappelant une recherche dichotomique, ce qui est **plus performant** que les listes, par exemple, que l'on doit parcourir élément par élément.
Ainsi la séquence {8 3 10 1 6 14 4 7 13} peut-être traduite sous la forme de l'arbre binaire de recherche suivant :



Exemple d'arbre binaire

a) Quelle est la séquence associée à l'arbre binaire de recherche ci-dessous ?

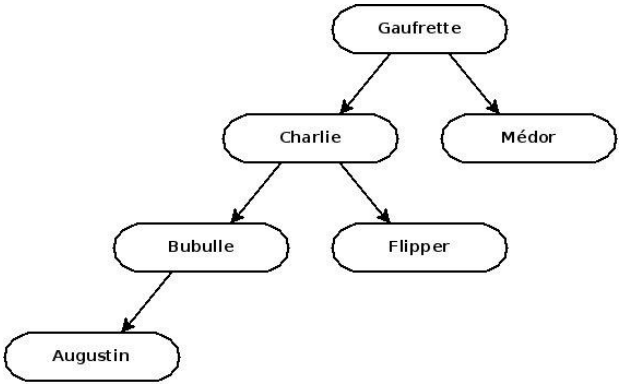


On obtient : {15 6 18 3 7 17 20 2 4 13 9}.

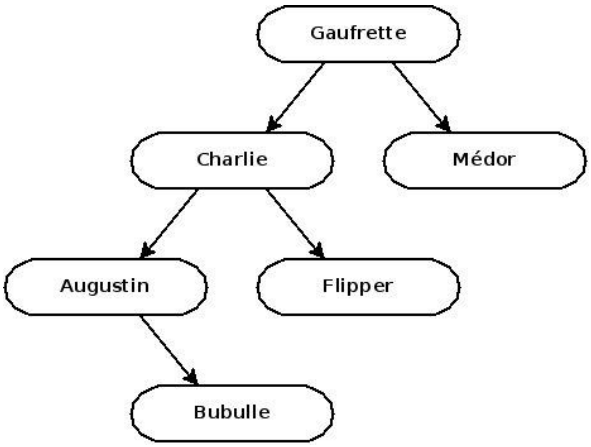
Autre exemple d'arbre binaire de recherche :

Un vétérinaire voudrait stocker les fiches médicales de ses patients, et, plutôt que d'utiliser un tableau ou une liste, on se propose d'utiliser un arbre binaire. La fiche contiendra différentes informations sur l'animal ; on utilisera son nom comme **clé** (c'est-à-dire comme critère pour la relation d'ordre), que l'on triera selon l'ordre alphabétique croissant. Le vétérinaire reçoit sa première patiente, qui répond au nom de **Gaufrette**. Comme sa fiche sera le premier nœud de notre arbre, elle en devient automatiquement la racine. Puis le vétérinaire reçoit les animaux dans l'ordre suivant afin de les soigner : Charlie, Médor, Flipper, Bubulle et Augustin.

b) Construire l'arbre binaire de recherche associé à cette séquence.



c) Imaginons que la séquence soit maintenant la suivante : {Gaufrette, Charlie, Médor, Flipper, Augustin, Bubulle}, c'est-à-dire que Augustin soit arrivé au rendez-vous médical avant Bubulle. L'arbre binaire de recherche est-il encore le même ? Quelle conclusion peut-on en tirer ?



L'arbre binaire de recherche n'est pas le même. On en conclu donc que la structure d'un arbre est dépendante de l'ordre dans lequel on insère les éléments !

2°) Parcours infixe d'un arbre binaire de recherche :

Un arbre binaire de recherche est donc un cas particulier d'arbre binaire. Pour avoir un arbre binaire de recherche :

- Il faut avoir un arbre binaire !
- Il faut que les clés de nœuds composant l'arbre soient ordonnables (on doit pouvoir classer les nœuds, par exemple, de la plus petite clé à la plus grande).
- Soit x un nœud d'un arbre binaire de recherche. Si y est un nœud du sous-arbre gauche de x, alors il faut que y.clé $\leqslant$ x.clé. Si y est un nœud du sous-arbre droit de x, il faut alors que x.clé $\leqslant$ y.clé.

a) Vérifiez que l'arbre de la question V) 1°) a) est bien un arbre binaire de recherche.

Pour chaque nœud x, on vérifie que pour chaque nœud y du sous-arbre de gauche, on a y.clé $\leqslant$ x.clé, et que pour chaque nœud z du sous-arbre de droite, on a x.clé $\leqslant$ z.clé.

b) Appliquez l'algorithme de parcours infixe à cet arbre. Que remarque-t-on ?

On obtient : 2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20. Les clés des nœuds sont donc classées par ordre croissant. C'est là une propriété remarquable des arbres binaires de recherche : **dans le cas d'un parcours infixe, les clés d'un arbre binaire de recherche sont renvoyées par ordre croissant**.

3°) Recherche d'une clé dans un arbre binaire de recherche :

Nous allons maintenant étudier un algorithme permettant de rechercher une clé de valeur k dans un arbre binaire de recherche. Si k est bien présent dans l'arbre binaire de recherche, l'algorithme renvoie vrai, dans le cas contraire, il renvoie faux.

■ Principe

La recherche dans un arbre binaire d'un nœud ayant une clé particulière est un procédé récursif. On commence par examiner la racine. Si sa clé est la clé recherchée, l'algorithme se termine et renvoie la racine. Si elle est strictement inférieure, alors elle est dans le sous-arbre gauche, sur lequel on effectue alors récursivement la recherche. De même si la clé recherchée est strictement supérieure à la clé de la racine, la recherche continue dans le sous-arbre droit. Si on atteint une feuille dont la clé n'est pas celle recherchée, on sait alors que la clé recherchée n'appartient à aucun nœud, elle ne figure donc pas dans l'arbre de recherche. On peut comparer l'exploration d'un arbre binaire de recherche avec la recherche par dichotomie qui procède à peu près de la même manière sauf qu'elle accède directement à chaque élément d'un tableau au lieu de suivre des liens. La différence entre les deux algorithmes est que, dans la recherche

dichotomique, on suppose avoir un critère de découpage de l'espace en deux parties que l'on n'a pas dans la recherche dans un arbre.

■ **Algorithme en pseudo-code :**

```
VARIABLE
T : arbre
x : noeud
k : entier

DEBUT
ARBRE-RECHERCHE(T,k) :
   si T == NIL :
      renvoyer faux
   fin si
   x ← T.racine
   si k == x.clé :
      renvoyer vrai
   fin si
   si k < x.clé :
      renvoyer ARBRE-RECHERCHE(x.gauche,k)
   sinon :
      renvoyer ARBRE-RECHERCHE(x.droit,k)
   fin si
FIN
```

a) Quelle est la particularité de la fonction ARBRE-RECHERCHE() ?

C'est une fonction récursive.

b) Appliquez l'algorithme de recherche d'une clé dans un arbre binaire de recherche sur l'arbre binaire de recherche de la question V) 1°) a). On prendra k = 13.

ARBRE-RECHERCHE(15,13)
x = 15
13 < 15 donc ARBRE-RECHERCHE(15.gauche,13)
x = 6
6 < 13 donc ARBRE-RECHERCHE(6.droit,13)
x = 7
7 < 13 donc ARBRE-RECHERCHE(7.droit,13)
x = 13 = k(13), donc renvoyer vrai.

c) Appliquez l'algorithme de recherche d'une clé dans un arbre binaire de recherche sur l'arbre binaire de recherche de la question V) 1°) a). On prendra k = 16.

ARBRE-RECHERCHE(15,16)
x = 16
16 > 15 donc ARBRE-RECHERCHE(15.droit,16)
x = 18
16 < 18 donc ARBRE-RECHERCHE(18.gauche,16)
x = 17
17 > 16 donc ARBRE-RECHERCHE(17.gauche,16)
17.gauche = NIL donc renvoyer faux.

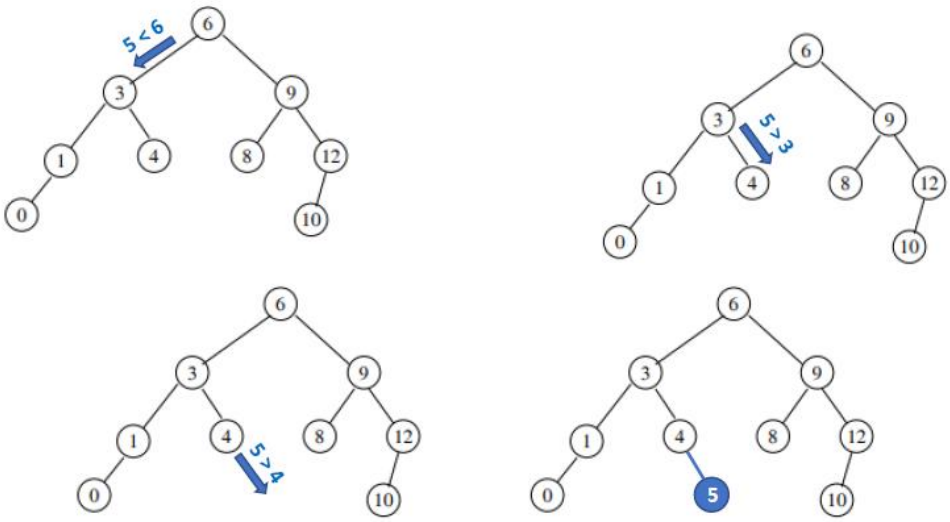
4°) Insertion d'une clé dans un arbre binaire de recherche :

Il est tout à fait possible d'insérer un nœud « y » dans un arbre binaire de recherche (non vide).

■ Principe

L'insertion d'un nœud commence par une recherche : on cherche la clé du nœud à insérer ; lorsqu'on arrive à une feuille, on ajoute le nœud comme fils de la feuille en comparant sa clé à celle de la feuille : si elle est inférieure, le nouveau nœud sera à gauche ; sinon il sera à droite.

Exemple : insertion de de la valeur 5 dans l'arbre de recherche binaire suivant :

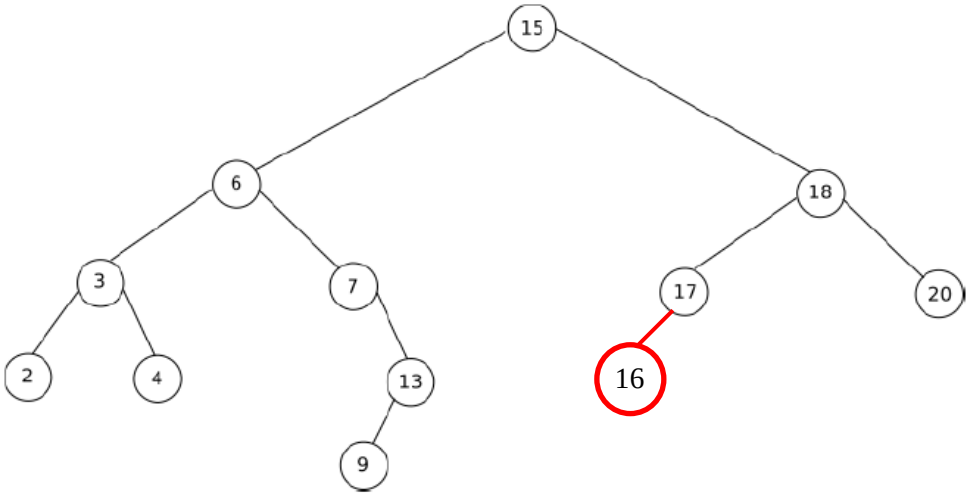


■ **Algorithme en pseudo-code :**

```
VARIABLE
T : arbre
x : noeud
y : noeud

DEBUT
ARBRE-INSERTION(T,y) :
   x ← T.racine
   tant que T ≠ NIL :
      x ← T.racine
      si y.clé < x.clé :
         T ← x.gauche
      sinon :
         T ← x.droit
      fin si
   fin tant que
   si y.clé < x.clé :
      insérer y à gauche de x
   sinon :
      insérer y à droite de x
   fin si
FIN
```

a) Appliquez l'algorithme d'insertion d'un nœud « y » dans un arbre binaire de recherche sur l'arbre de la question V) 1°) a). On prendra y.clé = 16.

ARBRE-INSERTION(T,16)
x = 15
15 < 16 donc T = 18
x = 18
16 < 18 donc T = 17
x = 17
16 < 17 et T = NIL
Donc insérer 16 à gauche de 17.
On obtient donc :



VI) IMPLEMENTATIONS PYTHON :

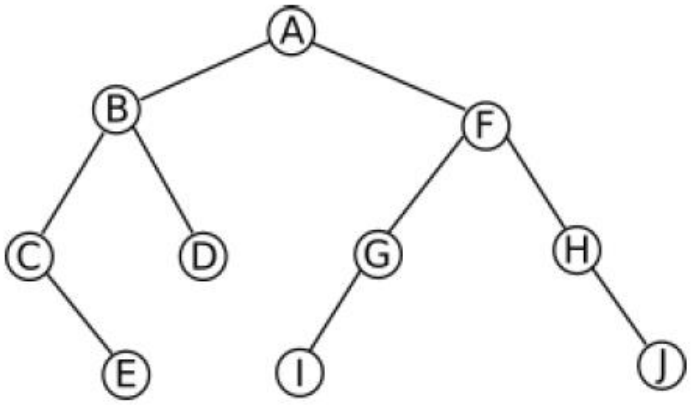
1°) Implémentation d'un arbre binaire en POO :

Python ne propose pas nativement de structure de données permettant d'implémenter directement les arbres binaires. Il va donc être nécessaire de créer manuellement cette structure. Pour programmer ce type de structure, nous allons utiliser le paradigme de la programmation orienté objet (P.O.O.).

Vous trouverez ci-dessous la classe "ArbreBinaire" qui va nous permettre d'implémenter des arbres binaires.

```
class ArbreBinaire:
    def __init__(self, valeur):
        self.valeur = valeur
        self.enfant_gauche = None
        self.enfant_droit = None

    def insert_gauche(self, valeur):
        if self.enfant_gauche == None:
            self.enfant_gauche = ArbreBinaire(valeur)
        else:
            new_node = ArbreBinaire(valeur)
            new_node.enfant_gauche = self.enfant_gauche
            self.enfant_gauche = new_node

    def insert_droit(self, valeur):
        if self.enfant_droit == None:
            self.enfant_droit = ArbreBinaire(valeur)
        else:
            new_node = ArbreBinaire(valeur)
            new_node.enfant_droit = self.enfant_droit
            self.enfant_droit = new_node

    def get_valeur(self):
        return self.valeur

    def get_gauche(self):
        return self.enfant_gauche

    def get_droit(self):
        return self.enfant_droit

    def estFeuille(self):
        if not self.enfant_gauche and not self.enfant_droit:
            return True
        else:
            return False
```

a) Complétez le programme précédent afin qu'il puisse construire l'arbre suivant à l'aide de la classe « ArbreBinaire ». On instanciera pour cela l'objet « racine » appartenant à la classe « ArbreBinaire ».




```
######début de la construction de l'arbre binaire###########

racine = ArbreBinaire('A')
racine.insert_gauche('B')
racine.insert_droit('F')

b_node = racine.get_gauche()
b_node.insert_gauche('C')
b_node.insert_droit('D')

f_node = racine.get_droit()
f_node.insert_gauche('G')
f_node.insert_droit('H')

c_node = b_node.get_gauche()
c_node.insert_droit('E')

g_node = f_node.get_gauche()
g_node.insert_gauche('I')

h_node = f_node.get_droit()
h_node.insert_droit('J')

######fin de la construction de l'arbre binaire###########
```

2°) Affichage d'un arbre binaire dans la console Python :

On peut afficher un arbre binaire dans la console Python en implémentant une fonction "graficarbre". Cette fonction a été développée à partir de Tkinter et permet une représentation graphique de l'arbre binaire.

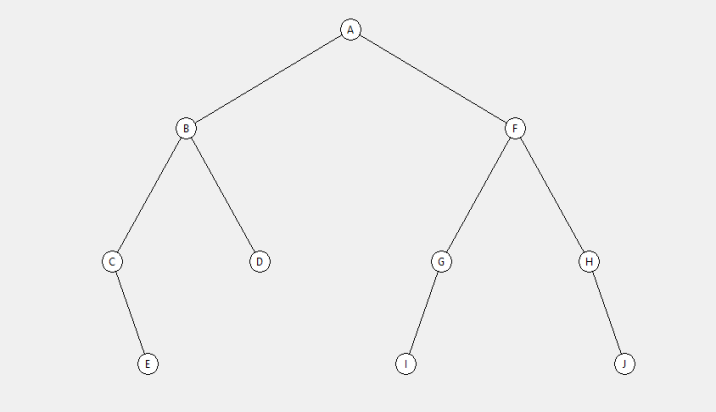
En voici le code, qu'il serait bon d'étudier.

```
# ==========================================================================
#                        Représentation graphique
# ==========================================================================
from tkinter import *
from math import *

# ------------------- cercle-------------------#
def cercle(canv,x,y,r,col,colf):
    """dessine un cercle graphique sur le canvas de centre (x,y) de rayon r de
    couleur col et de couleur de fond colf"""
    canv.create_oval(x-r,y-r,x+r,y+r,outline=col, fill=colf)
# ------------------- dessinenoeud-------------------#
def dessinenoeud(canv,x,y,r,noeud):
```

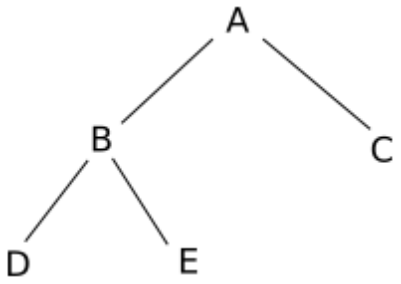
```
    """ dessine un noeud graphique : un cercle rempli avec la valeur du noeud"""
    cercle(canv,x,y,r,"black","white")
    canv.create_text(x,y,text=noeud.valeur,fill="black")
# ------------------- centresuivant-------------------#
def centresuivant(x,y,r,m,dir,ouverture):
    # ouverture permet d'ajuster l'angle d'ouverture des noeuds
    """ calcule la position de noeud suivant :
        on calcule a et b les décalages par rapport à la position actuelle x,y
        dir permet de spécifier:
        si c'est un fils gauche on retranche le a
        si c'est un fils droit on ajoute le a à x
        pour y on ajoute toujours b dans les deux cas.
        m la distance entre les cercles : le nœud et ses descendants.
    """
    a=(2*r+m)*sin(pi/(ouverture))   # calcule le décalage sur l'axe des x : coordonnées polaire vers
                                    # cordonnées cartésiennes
    b=(2*r+m)*cos(pi/(ouverture))   # de même pour le décalage sur y. l'angle d'ouverture est 45 ° pour ouverture = 4
    if dir=="l":            # dir pour left ou right c.à.d. fils gauche ou fils droit
        x1,y1=x-a,y+b       # on décale vers la gauche donc on retranche a de x et
                            # on ajoute b à y on descend vers le bas
    else:
        x1,y1=x+a,y+b       # on décale vers la droite donc on ajoute a à x
    ouverture += 3          # en augmentant ouverture, on diminue l'angle pour la ligne suivante
    return x1,y1,ouverture
# ------------------- tracearbre-------------------#
def tracearbre(canv,x,y,r,m,noeud,ouverture):
    """ trace l'arbre graphique récursivement"""
    pas = 50                # pas ajustable permettant de réduire la distance entre les noeuds
                            # pour éviter que des noeuds se superposent
    if noeud.estFeuille()==False:    # si le noeud n'est pas une feuille
        if noeud.enfant_droit==None and noeud.enfant_gauche!=None :       # s'il a un fils gauche mais pas de fils droit
            x1,y1,ouverture=centresuivant(x,y,r,m,"l",ouverture)          # récupération de la position du noeud fils
            canv.create_line(x,y,x1,y1,fill="black") # tracé d'une droite entre x,y et x1,y1
                                                     # cette fonction est dans la bibilothèque tkinter
            tracearbre(canv,x1,y1,r,m-pas,noeud.enfant_gauche,ouverture)  # appel récursif pour traiter ce fils
        elif noeud.enfant_droit!=None and  noeud.enfant_gauche==None :    # s'il a un fils droit mais pas de fils gauche
            x1,y1,ouverture=centresuivant(x,y,r,m,"r",ouverture)          # récupération de la position du noeud fils
            canv.create_line(x,y,x1,y1,fill="black")    # tracé d'une droite entre x,y et x1,y1
            tracearbre(canv,x1,y1,r,m-pas,noeud.enfant_droit,ouverture)   # appel récursif pour traiter ce fils
        else:                                   # si il a un fils gauche et un fils droit
            x1,y1,ouverture=centresuivant(x,y,r,m,"l",ouverture)          # récupération de la position du noeud fils gauche
            canv.create_line(x,y,x1,y1,fill="black")    # tracé d'une droite entre x,y et x1,y1
            tracearbre(canv,x1,y1,r,m-pas,noeud.enfant_gauche,ouverture)  # appel récursif pour traiter ce fils gauche
            x1,y1,ouverture=centresuivant(x,y,r,m,"r",ouverture-3)        # récupération de la position du noeud fils droit
                                                        # ouverture-3 pour compenser sur le noeud de droite
                                                        # le +=3 dans centresuivant déjà appliqué
            canv.create_line(x,y,x1,y1,fill="black")    # tracé d'une droite entre x,y et x1,y1
            tracearbre(canv,x1,y1,r,m-pas,noeud.enfant_droit,ouverture)  # appel récursif pour traiter ce fils droit
    dessinenoeud(canv,x,y,r,noeud)                  # tracé du noeud courant

# ------------------- graphicarbre-------------------#
def graphicarbre(noeud):
    """ fonction de tracé graphique d'un arbre """
    cwidth=1000             # la largeur du canvas graphique
    cheight=700             # la hauteur du canvas
    couleurs=["red","green","bleu","white","black","cyan","magenta","yellow"]
    # fen est l'objet fenêtre héritée de la bibliothèque tkinter
    fen=Tk()
    # création d'un bouton avec la commande fermer(quit) attachée à la fenêtre
```

```
btn=Button(fen, text="Quitter",command=fen.destroy)
# placement du bouton en bas de la fenêtre
btn.pack(side="bottom")
# création d'un panneau dans lequel on affichera l'arbre
pan=LabelFrame(fen)
# placement de ce panneau en faut de la fenêtre
pan.pack(side="top")
# creation du canva graphique dans ce panneau
canv=Canvas(pan,width=cwidth,height=cheight)
# placement de ce canva en haut du panneau graphique
canv.pack(side="top")
# appel de la fonction de tracé graphique de l'arbre créé ci haut
tracearbre(canv,cwidth//2,100,12,200,noeud,3)
# actualisation de l'affichage graphique
fen.mainloop()
```

a) Implémenter le programme complet et vérifier que l'arbre binaire construit est correct.
On obtient bien, après avoir ajouté : graphicarbre(racine) à la fin du programme.



b) Programmez à l'aide de la classe "ArbreBinaire", l'arbre binaire suivant et vérifier avec la fonction affiche que le programme donne bien le résultat attendu :




```
######début de la construction de l'arbre binaire###########

racine = ArbreBinaire('A')
racine.insert_gauche('B')
racine.insert_droit('C')

b_node = racine.get_gauche()
b_node.insert_gauche('D')
b_node.insert_droit('E')

######fin de la construction de l'arbre binaire###########

graphicarbre(racine)
```